

Bootcamp Databricks

Engenharia de Dados dentro da Plataforma Data & AI

I&D Capgemini Brasil, Setembro/2026

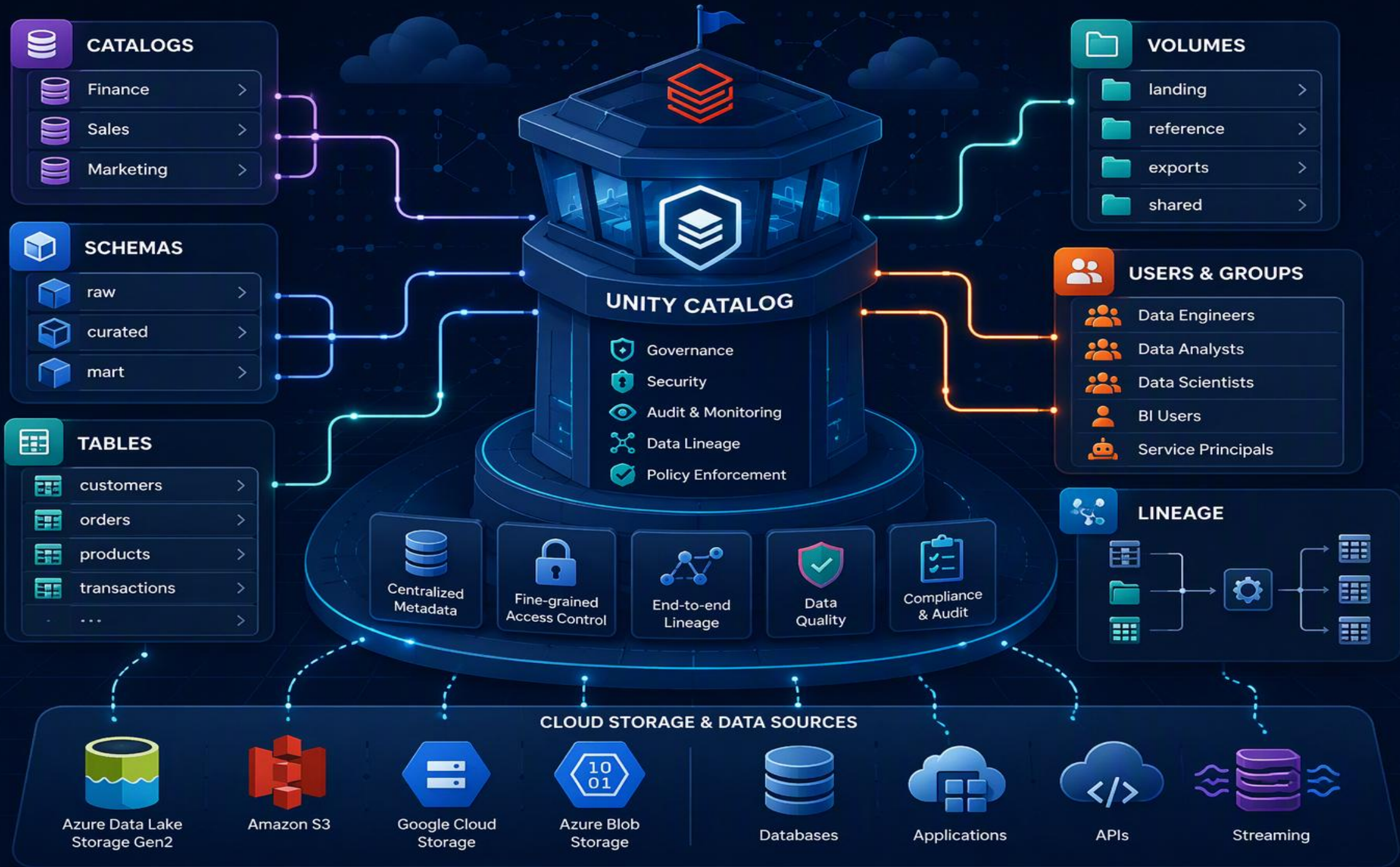
Agenda

Aulas	Assunto
Dia 1 – 14/09/2026	Fundamentos de Engenharia de dados e Databricks
Dia 2 – 15/09/2026	SQL, bancos, tabelas e Delta Lake
Dia 3 – 16/09/2026	Unity Catalog, schemas, volumes e governança
Dia 4 – 17/09/2026	Ingestão de dados: batch, incremental, Auto Loader
Dia 5 – 18/09/2026	ETL vs ELT & transformação arquitetura medalhão
Dia 6 – 21/09/2026	Modelagem de dados para Analytics
Dia 7 – 22/09/2026	Processamento distribuído com Spark no Databricks
Dia 8 – 23/09/2026	Streaming e processamento contínuo
Dia 9 – 24/09/2026	Orquestração, Jobs, DataOps e boas práticas
Dia 10 – 25/09/2026	Qualidade, observabilidade, arquitetura e desafio final

Dia 3 - Unity Catalog, schemas, volumes e governança

Organização e governança de dados no Lakehouse

I&D Capgemini Brasil, Setembro/2026



Objetivos do Dia 3

Ao final do encontro, você deve entender como organizar e governar dados no Databricks.

- Explicar o papel do Unity Catalog na governança do Lakehouse.
- Diferenciar metastore, catálogo, schema, table, view e volume.
- Aplicar a convenção catalog.schema.table em consultas e criação de objetos.
- Entender managed vs external tables e managed vs external volumes.
- Executar um laboratório criando namespaces, tabela e volume.

Mensagem central

Dados sem governanca viram risco operacional, risco de seguranca e retrabalho para engenharia.

Foco prático

Organizar objetos do lakehouse como produto: nome, dono, permissao, localizacao e proposito.

Recap do Dia 2: tabela também é contrato

O Dia 3 aprofunda a pergunta: onde a tabela vive, quem pode acessar e quem responde por ela?



- Sem namespace claro, objetos viram bagunça.
- Sem ownership, ninguém responde por qualidade e acesso.
- Sem privilégios mínimos, segurança vira exceção manual.
- Sem volumes, arquivos ficam fora do modelo de governança.

Por que Governança?

Bootcamp Databricks

I&D Capgemini Brasil

O problema: dados sem governo nao escalam

O crescimento da plataforma exige padrão de organização, acesso e rastreabilidade.

Silos	Acesso	Busca	Auditoria
Times criam objetos sem padrão comum.	Permissões viram exceções manuais.	Usuários não sabem qual dado e confiável.	Difícil explicar origem, uso e dono.

Governança e o conjunto de controles que permite operar dados em escala sem perder segurança, clareza e responsabilidade.

Organizar: hierarquia e nomenclatura.

Proteger: privilégios e grupos.

Documentar: metadados, comentários e tags.

Rastrear: lineage, auditoria e ownership.

Unity Catalog: backbone de governança

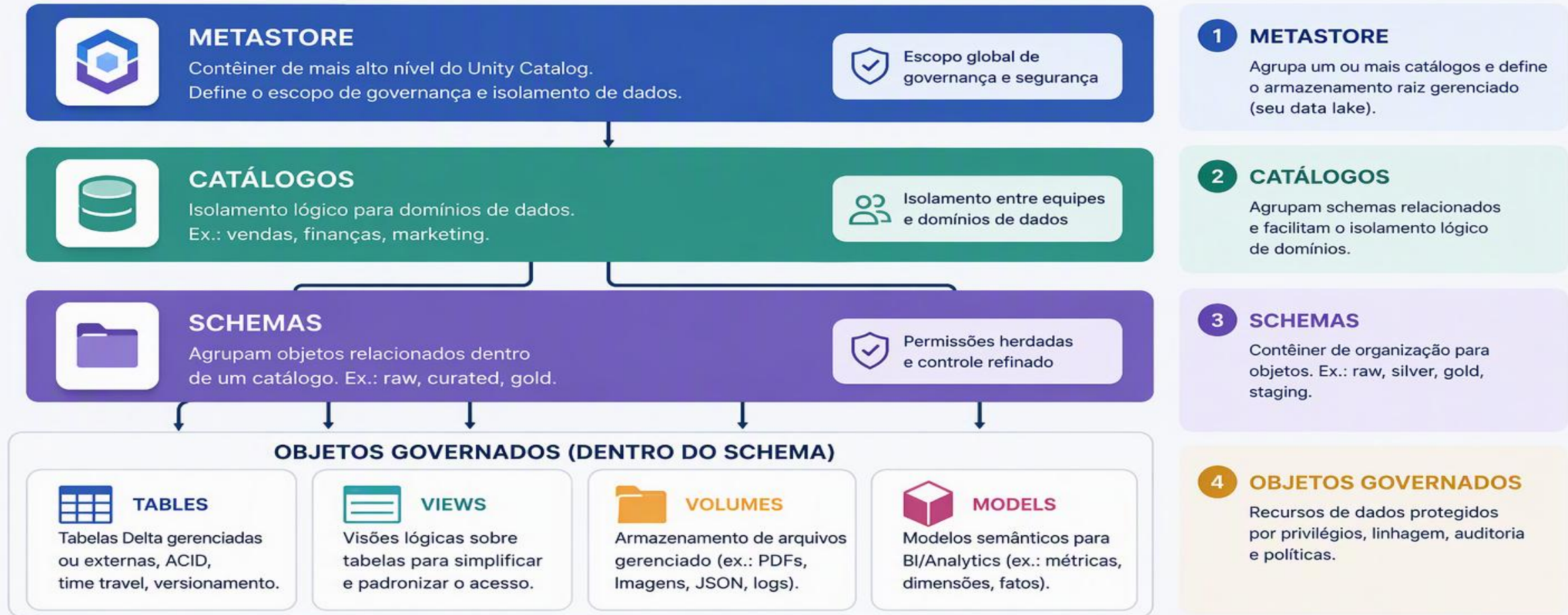
Um ponto central para organizar, proteger e descobrir ativos de dados e IA.



- Metadados centralizados para objetos governados.
- Modelo de privilégios aplicado a objetos securizáveis.
- Organização por metastore, catálogos e schemas.
- Controle sobre tabelas, views, volumes, funções e modelos.
- Base para lineage, auditoria e descoberta de dados.

UNITY CATALOG – HIERARQUIA E GOVERNANÇA

Organize, governe e dê acesso seguro aos dados em toda a sua plataforma Databricks



GOVERNANÇA E SEGURANÇA TRANSVERSAIS



CONTROLE DE ACESSO

Privilégios granulares por usuário, grupo ou service principal.



LINHAGEM

Rastreie origem, uso e transformações dos dados.



AUDITORIA

Logs de acesso e alterações para conformidade.



POLÍTICAS

Classificação, tags e políticas de dados aplicadas.



SEGURANÇA

Integração com Azure AD, criptografia e isolamento de dados.



UNIDADE • GOVERNANÇA • SEGURANÇA • ESCALABILIDADE • COLABORAÇÃO

Metastore, catálogo e schema: responsabilidades

Separar responsabilidades evita misturar ambiente, domínio e camada.

Metastore

Escopo mais alto de governança.
Agrupar catálogos e define o domínio de metadados da plataforma.

Catálogo

Unidade lógica para ambiente, domínio ou produto de dados.
Ex.: dev_catalog, prd_catalog.

Schema

Container de objetos dentro do catálogo. Ex.: billing_gold, finance_silver.

Padrão recomendado para o treinamento

```
catalog.schema.object
```

```
capgemini_academy.bronze.orders
```

```
capgemini_academy.gold.fato_faturamento
```

- Aprenda a sempre localizar o objeto pelo namespace completo.
- Evite depender de contexto implícito de catálogo/schema no notebook.
- Use nomes que revelem ambiente, domínio e camada.

Design de namespace para projetos reais

Um bom nome reduz ambiguidade antes mesmo de aplicar permissão.

Exemplo de convenção

```
<ambiente>_catalog.<dominio>_<camada>.<objeto>
```

```
dev_catalog.sales_bronze.orders_raw  
stg_catalog.sales_silver.orders_clean  
prd_catalog.sales_gold.fct_revenue
```

Ambiente

dev, stg/qa, prd

Domínio

sales, finance, billing, supply

Camada

bronze, silver, gold, mart

Regra de ouro: se o nome não explica propósito, alguém vai usar errado.

Objetos governados

Bootcamp Databricks

I&D Capgemini Brasil

Objetos governados no Unity Catalog

Você precisa diferenciar tabela, view e volume antes do laboratório.

Tables

Dados tabulares consultáveis por SQL. Podem ser managed ou external em Unity Catalog.

Views

Camada lógica de consulta. Simplifica acesso, padroniza consumo e pode esconder complexidade.

Volumes

Governam arquivos não tabulares ou arquivos usados por pipelines, como CSV, JSON, imagens e logs.

Mensagem Importante

- Tabela é para dados estruturados.
- Volume é para arquivos.
- View é para expor uma lógica de consulta.
- Todos precisam de dono, permissão e propósito.

Exemplo rápido

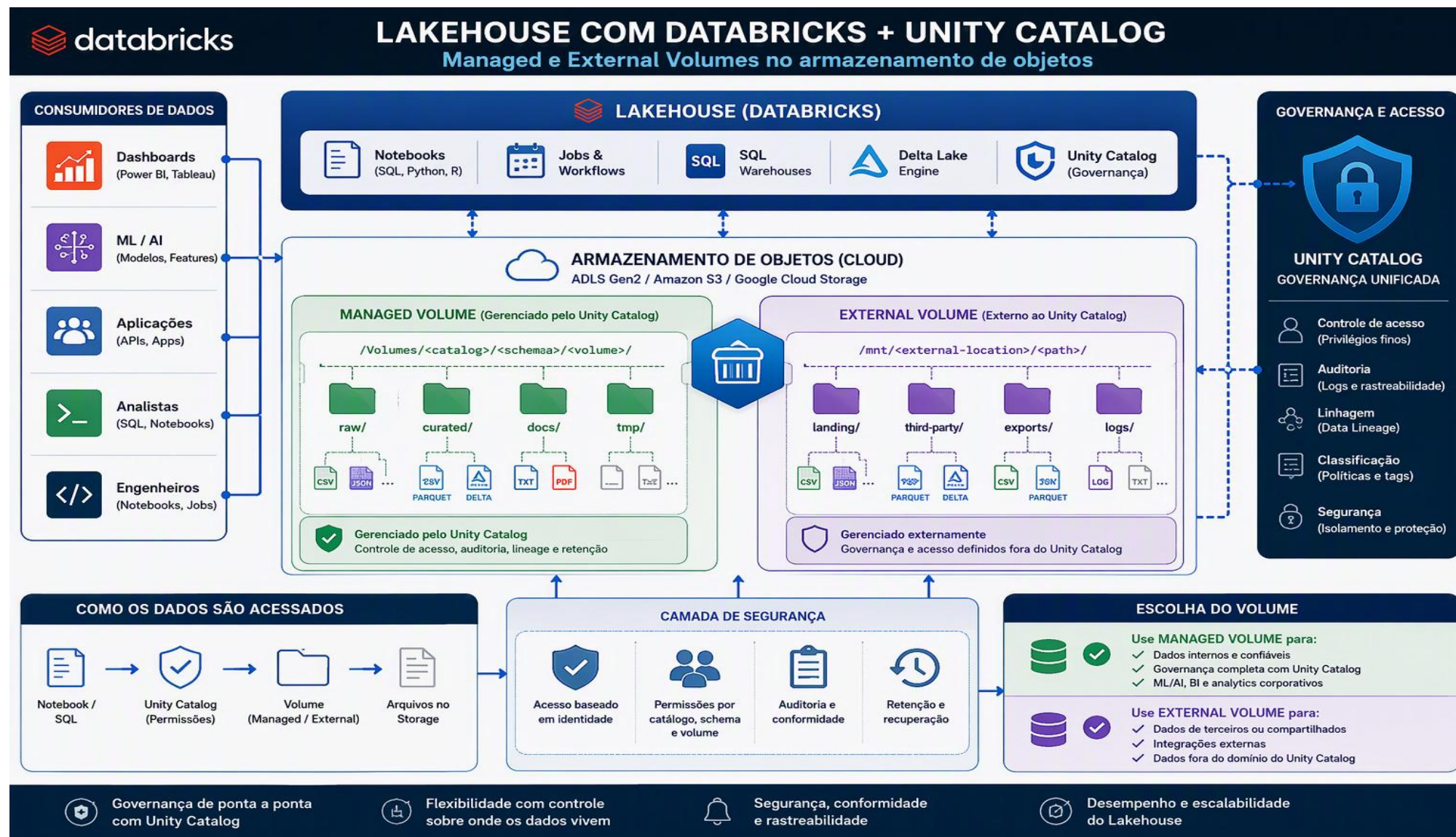
```
bronze.orders_raw = tabela
```

```
landing = volume
```

```
vw_orders_today = view
```

Volumes: arquivos também precisam de governança

Volumes permitem trabalhar com arquivos dentro do modelo de governança do Unity Catalog.



Managed vs external: escolha consciente

O tipo de objeto impacta a localização, responsabilidade e permissão.

Tipo	Quando usar	Ponto de atenção
Managed table/volume	Quando a plataforma deve gerenciar localização e ciclo de vida.	Menos liberdade de path, mais padrão.
External table/volume	Quando dados já vivem em local externo controlado.	Exige estratégia clara de external location e acesso.

Para iniciantes

Comece com managed quando o objetivo for aprender organização e consumo.

Para projetos

External aparece quando há storage corporativo, zonas existentes e integração com legado.

Para governança

Não deixe external virar atalho para burlar controle de acesso.

Modelo de privilégios: mínimo necessário

Unity Catalog trabalha com objetos securizáveis, principais e privilégios.

Principal

Usuário, grupo ou service principal que recebe permissão.

Objeto securizável

Metastore, catalogo, schema, table, view, volume e outros.

Privilégio

A ação permitida, como USE CATALOG, USE SCHEMA, SELECT, READ VOLUME.

Exemplo didático

```
GRANT USE CATALOG ON CATALOG dev_catalog TO 'data_engineers';  
  
GRANT USE SCHEMA ON SCHEMA dev_catalog.sales_bronze TO 'data_engineers';  
  
GRANT SELECT ON TABLE dev_catalog.sales_bronze.orders_raw TO 'data_analysts';  
  
GRANT READ VOLUME ON VOLUME dev_catalog.sales_bronze.landing TO 'data_engineers';
```

Permissões para volumes

Volumes introduzem privilégios específicos para trabalhar com arquivos.

Ação	Catálogo	Schema	Volume
Ler/listar arquivos	USE CATALOG	USE SCHEMA	READ VOLUME
Criar/alterar arquivos	USE CATALOG	USE SCHEMA	READ VOLUME + WRITE VOLUME
Criar volume gerenciado	USE CATALOG	USE SCHEMA + CREATE VOLUME	-
Criar volume externo	USE CATALOG	USE SCHEMA + CREATE VOLUME	CREATE EXTERNAL VOLUME na external location

- Owners recebem privilégios sobre o volume.
- Permissões READ VOLUME e WRITE VOLUME podem ser concedidas no catálogo ou schema para cascadear.

Ownership: quem responde pelo objeto?

Governança efetiva exige dono claro e preferencialmente grupos, não indivíduos.

Owner

Cada objeto tem um owner.
Owner pode conceder acesso e administrar o objeto.

MANAGE

Permite gerenciar privilégios no objeto sem necessariamente ser owner.

Grupo

Boa prática: ownership em grupos reduz dependência de pessoas específicas.

Nunca desenhe governança baseada em usuários isolados quando o processo exige continuidade operacional.

Demo e laboratório

Bootcamp Databricks

I&D Capgemini Brasil

Demo guiada: Catalog Explorer + SQL

Mostre a organização antes de codificar.

Catalog Explorer

- Abrir Catalog.
- Localizar o catálogo do workspace.
- Explorar schemas existentes.
- Abrir tabela do Dia 2.
- Mostrar aba de permissões, detalhes e lineage quando disponível.

SQL Notebook

- Usar nome completo `catalog.schema.table`.
- Criar schema de laboratório.
- Criar volume gerenciado.
- Criar tabela simples.
- Consultar e validar objetos.

Script de demo: namespace e objetos

Adapte nomes conforme o catálogo permitido no workspace do treinamento.

```
-- 1. Definir contexto
USE CATALOG capgemini_academy;

-- 2. Criar schema de laboratorio
CREATE SCHEMA IF NOT EXISTS treinamento_dia3;

-- 3. Criar volume gerenciado
CREATE VOLUME IF NOT EXISTS capgemini_academy.treinamento_dia3.landing;

-- 4. Criar tabela Delta simples
CREATE TABLE IF NOT EXISTS capgemini_academy.treinamento_dia3.clientes_demo (
  id INT,
  nome STRING,
  cidade STRING,
  data_cadastro TIMESTAMP
) USING DELTA;

-- 5. Validar
SHOW TABLES IN capgemini_academy.treinamento_dia3;
SHOW VOLUMES IN capgemini_academy.treinamento_dia3;
```

Laboratório do aluno: entregável do Dia 3

Ao final, você deve sair com um ambiente lógico organizado.

Databricks

Diá 2 – SQL, Bancos, Tabelas e Delta Lake

Laboratório guiado – Criar e consultar uma tabela Delta com Unity Catalog

NotebookCatalogJobsDE

NewWorkspaceRecentsCatalogWorkflowsComputeSQLData EngineeringMachine Learning

Catalog Explorer

AllCatalogsSchemasVolumes

workspace (Current metastore)

dev

Schemas

bronze

silver

gold

Volumes

data

stage

analytics

Schemas

mart

samples

dev (Catalog)

Comentário

Catálogo para desenvolvimento.

Proprietário

data.admins

02_Sql_Delta_LakePython

Run allCluster SQL WarehouseSave

%sql-- 1. Criar schemaCREATE SCHEMA IF NOT EXISTS dev.bronze;(1) Spark JobsCommand executed

%sql-- 2. Criar tabela Delta gerenciadaCREATE TABLE IF NOT EXISTS dev.bronze.clientes (id INT,nome STRING,cidade STRING,data_cadastro TIMESTAMP)USING DELTATBLPROPERTIES (delta.feature.appendOnly = 'true');> (1) Spark JobsCommand executed

%sql-- 3. Inserir dadosINSERT INTO dev.bronze.clientes VALUES(1, 'Ana Silva', 'São Paulo', current_timestamp()),(2, 'Bruno Lima', 'Rio de Janeiro', current_timestamp()),(3, 'Carla Dias', 'Belo Horizonte', current_timestamp());> (2) Spark JobsCommand executed

%sql-- 4. Consultar dadosSELECT * FROM dev.bronze.clientes;

id	nome	cidade	data_cadastro
1	Ana Silva	São Paulo	2024-05-01 10:15:23.123
2	Bruno Lima	Rio de Janeiro	2024-05-01 10:15:23.125
3	Carla Dias	Belo Horizonte	2024-05-01 10:15:23.126

SQL WarehouseReady3.21s2: PythonAuto

Permissões (Unity Catalog)

Princípio

data.engineers

Tipo

Grupo

Conceder nas seguintes áreas

☒ Catalog: workspace☒ Schema: dev☒ Tabela: dev.bronze.clientes

Privilegios

☒ USE CATALOG☒ USE SCHEMA☒ SELECT☐ MODIFY☐ CREATE TABLE☐ DROP☐ MANAGE

CancelarConceder

Namespace

Todos os objetos são organizados em:

catalog.schema.table

Catálogo

Nível mais alto

Schema

Agrupar objetos relacionados

Tabela

Objeto de dados consultável

Não esqueça

☒ Use tabelas Delta (ACID, Time Travel e Schema Enforcement)☒ Prefira tabelas gerenciadas pelo Unity Catalog☒ Organize por camadas: bronze, silver, gold☒ Sempre valide com SELECT após cada passo☒ Mantenha boas práticas de segurança e governança

Onde estamos na arquitetura

Você está aqui

FontesIngestãoArmazenamento (Data Lake)TransformaçãoConsumo

Tipos de tabela no Unity Catalog

Managed (gerenciada)

Dados armazenados e gerenciados pelo DatabricksRecomendado

External (externa)

Dados em storage externoRequer cuidado com permissões

Arquivos Delta Lake

_delta_log

00000000000000000000000000000000.parquet

...

data

part-00000-xxxxx.parquet

part-00001-xxxxx.parquet

...

☒ ACID Transactions☒ Time Travel☒ Schema Enforcement☒ Auditável e Confiável

Entregável

Schema, tabela e volume criados e documentados no notebook.

- Criar ou usar um catálogo autorizado.
- Criar schema do laboratório.
- Criar volume gerenciado para landing.
- Criar tabela sob namespace completo.
- Executar consultas de validação.
- Registrar no notebook o que cada objeto representa.

Laboratório - Parte 1: criar schema e tabela

Ponto pedagógico: tudo deve estar em namespace qualificado.

```
-- Ajuste o catalogo conforme o ambiente
USE CATALOG capgemini_academy;

CREATE SCHEMA IF NOT EXISTS treinamento_dia3;

CREATE TABLE IF NOT EXISTS capgemini_academy.treinamento_dia3.departamentos (
  id_departamento INT,
  nome_departamento STRING,
  area STRING,
  data_criacao TIMESTAMP
) USING DELTA;

INSERT INTO capgemini_academy.treinamento_dia3.departamentos VALUES
(1, "Dados", "Tecnologia", current_timestamp()),
(2, "Financeiro", "Corporativo", current_timestamp()),
(3, "Operacoes", "Negocio", current_timestamp());
```

Laboratório - Parte 2: criar volume e organizar arquivos (opcional)

Ponto pedagógico: volume e lugar governado para arquivos.

```
CREATE VOLUME IF NOT EXISTS capgemini_academy.treinamento_dia3.landing;

-- Explorar volumes pelo Catalog Explorer
SHOW VOLUMES IN capgemini_academy.treinamento_dia3;

-- Caminho padrao para volume no Databricks:
-- /Volumes/<catalog>/<schema>/<volume>/

-- Opcional: use o Catalog Explorer para carregar um pequeno CSV
-- no volume landing e depois registre o caminho no notebook.
```

Laboratório - Parte 3: validar governança

Validar é tão importante quanto criar.

```
-- Validar objetos
SHOW SCHEMAS IN capgemini_academy;
SHOW TABLES IN capgemini_academy.treinamento_dia3;
SHOW VOLUMES IN capgemini_academy.treinamento_dia3;

DESCRIBE TABLE EXTENDED capgemini_academy.treinamento_dia3.departamentos;
SELECT area, COUNT(*) AS qtd
FROM capgemini_academy.treinamento_dia3.departamentos
GROUP BY area
ORDER BY qtd DESC;

-- Opcional para demonstracao pelo instrutor
SHOW GRANTS ON SCHEMA capgemini_academy.treinamento_dia3;
```

Quiz rápido de fixação

Perguntas para reforçar o modelo mental antes do encerramento.

- 1) Qual a diferença entre catálogo e schema?
- 2) Quando usar volume em vez de tabela?
- 3) Por que catalog.schema.table reduz risco operacional?
- 4) Qual privilégio é necessário para consultar uma tabela?
- 5) Por que ownership em grupo é melhor que um usuário individual?

Fechamento: o que muda depois do Dia 3

Agora você está sabendo organizar objetos antes de construir pipelines maiores.

Antes

Criar tabela

Você sabia criar uma tabela e consultar dados.

governar

Depois

Criar ativo governado

O aluno sabe onde o objeto vive, quem acessa, qual o propósito e como validar.

DELETAR todos os objetos criados anteriormente, schema treinamento_dia3.

Referências de datasets abertos para prática

O objetivo é capturar dados e disponibilizá-los no Unity Catalog para trabalhos futuros.

Dataset	Domínio	Complexidade
MovieLens	Entretetimento	Baixa
Olist E-commerce (Brasil)	Varejo	Média
Airbnb Open Data	Hospedagem	Média
NYC Taxi Trips	Mobilidade	Alta
Chicago Crimes	Governo	Média
Formula 1 Dataset	Esporte	Média
Spotify Dataset	Música	Média
Stack Overflow Survey	Tecnologia	Média
Retail Sales Dataset	Varejo	Alta
Covid Open Data	Governo	Alta
Dados Financeiros (Yahoo)	Financeiro	Alta

Bootcamp Databricks

Laboratório Dia 3

Unity Catalog + Schemas + Volumes + Governança

Objetivo: Governar os ativos criados nos Dias 1 e 2, aplicando namespace completo, criando schema e volume governados, documentando objetos e entendendo acesso, ownership e propósito operacional.

Narrativa: no Dia 1 criamos a fundação. No Dia 2 criamos tabelas analíticas. No Dia 3 transformamos objetos em ativos governados.

Jornada prática do laboratório

1

CAT

Catalog Explorer
Abrir o catálogo capgemini_academy e localizar bronze, silver, gold, tabelas e detalhes.

2

NS

Namespace
Praticar catalog.schema.object com objetos já criados nos dias anteriores.

3

SCH

Schema governance
Criar schema **governance** como área lógica de governança.

4

VOL

Volume governado
Criar governance.landing_v2 para arquivos usados em ingestão futura.

5

VIEW

View governada
Criar vw_top_movies sobre gold.topRated_movies para padronizar consumo.

6

OWN

Ownership
Discutir quem responde por Bronze, Silver, Gold e volumes.

7

VAL

Validação
Executar SHOW, DESCRIBE, SELECT e, se possível, SHOW GRANTS.

8

LAB

Mini desafio
Criar vw_comedy_movies e explicar diferença entre table, view e volume.

Entregável final

capgemini_academy

BRONZE

movies ratings

➔

GOVERNANCE

landing_v2 views

➔

GOLD

topRated_movies

Checklist do laboratório

✔

Catalog Explorer explorado

✔

Namespace completo aplicado

✔

Schema governance criado

✔

Volume landing_v2 criado

✔

Views governadas criadas

✔

Diferenças table/view/volume explicadas

Comandos centrais

```
USE CATALOG capgemini_academy;
CREATE SCHEMA IF NOT EXISTS governance;
CREATE VOLUME IF NOT EXISTS governance.landing_v2;
CREATE OR REPLACE VIEW governance.vw_top_movies AS
SELECT * FROM gold.topRated_movies;
SHOW VOLUMES IN capgemini_academy.governance;
```

Governança não é burocracia. É engenharia aplicada à confiança dos dados.



Desafio do Dia 3 – Assistir e resumir

- Databricks Unity Catalog
<https://www.youtube.com/watch?v=Cbf6Fs93Jl4>
- Como aprender Databricks rapidamente
<https://www.youtube.com/watch?v=USKnnzUvKM4>

Thank you.

Make it real.

Capgemini 